Backend Take-Home Task

Full-Stack Developer · expected effort 5–7 hours · deadline 24 hours from when you receive this

> **What we are testing.** This task has no single correct answer. Several designs can work, and each one breaks in a different way. We want to see which design you choose, why you choose it, what you give up, and whether it holds when things fail.
>
> **AI is expected.** We use Claude Code every day and assume you will too. Part of your score comes from how you set up the project so the AI works safely and produces code you can trust (section 5).

1. Context

We run a food-ordering platform for restaurants in Egypt. Customers order and pay on their phones. Many restaurants use a POS (point-of-sale) system at the counter, and every paid order has to appear on that POS so the kitchen can prepare it.

Our API runs as **several identical instances** behind a load balancer. Any instance can be restarted or killed at any moment during a deploy. The POS belongs to a third party: it is slow, it fails sometimes, and we cannot change it.

2. The problem

Build the part of the backend that **sends orders to the restaurant's POS** when a check is paid, and cancels them on the POS when a check is cancelled.

A *check* is our word for an order. You only need these endpoints on our side:

```
POST /checks                 create a check (restaurantId, items[], total)  → status
"open"
POST /checks/:id/pay         mark it paid (simulate the payment, no gateway)
POST /checks/:id/cancel      cancel it (allowed while open or paid)
GET  /checks/:id/sync        where this check is in the POS sync, and the last error if
any
POST /checks/:id/sync/retry  manually retry a check whose sync has stopped
```

Guarantees your design must provide

1. **Paying is never blocked by the POS.** `POST /checks/:id/pay` responds in under 300 ms and succeeds even when the POS is down or slow.
2. **Exactly one POS order per paid check.** Every paid check eventually reaches the POS. None is lost and none is duplicated, even when:
   - an API instance is killed at *any* point (before, during or after any step),
   - the POS creates the order but the response times out, so you do not know whether it succeeded,
   - the POS returns 500 or 429,
   - two or more API instances are running at the same time.

3. **Order of events per check.** A cancel must never reach the POS before the create it cancels. If a check is cancelled before its create was ever sent, the POS must end up with no open order for it. How you achieve that is up to you, but you must explain it.
4. **Respect the POS rate limit.** At most **5 requests per 10 seconds per restaurant**, counted across *all* API instances together. One very busy restaurant must not delay orders for other restaurants.
5. **Failures are visible and stop when they should.** Temporary errors are retried. A permanent rejection (POS returns 422) stops retrying, shows the reason in `GET /checks/:id/sync`, and can be retried by hand with the retry endpoint.

3. The fake POS (you build it, keep it small)

Write a small fake POS as a separate process or a separate Nest app in the same repo. It must behave like this:

```
POST /orders                { externalRef, restaurantId, items, total }  → 201 {
posOrderId }
GET  /orders?externalRef=... → list of POS orders with that externalRef
POST /orders/:posOrderId/cancel → 200
GET  /debug/orders          → every order it holds (used by your tests)
```

- **It is not idempotent.** Sending the same `externalRef` twice creates two orders. That is how many real POS systems behave.
- **Failure modes**, each set by an environment variable with a rate between 0 and 1:
  - `FAIL_RATE`: returns 500 without creating anything,
  - `TIMEOUT_AFTER_CREATE_RATE`: creates the order, then waits 15 seconds before responding,
  - `SLOW_RATE`: answers correctly after 3–8 seconds.
- Returns **429** when a restaurant goes over 5 requests in 10 seconds.
- Returns **422** for any check with `total <= 0` (a permanent rejection).

4. Stack and deliverables

Required stack

TypeScript, **NestJS**, **PostgreSQL** with **Drizzle ORM** (schema plus generated migrations). Redis and/or BullMQ only if your design needs them; using them is not a plus by itself. Everything must start with `docker compose up`, and the compose file must run **at least two API instances**.

Deliverables (one private GitHub repo, shared with us)

1. **The code** for the API and the fake POS.
2. **`DECISIONS.md`**, at most 2 pages:
   - at least **three** different designs you considered for getting checks to the POS, with what each one gets right and where each one breaks,
   - the design you chose, and why it beats the others *for this problem*,
   - how you handle the "timed out but maybe created" case, and what risk is left,
   - what you would change at 100× the volume, and what you would do with another day.

3. **A chaos test**: one command (for example `pnpm test:chaos`) that, with every POS failure mode switched on and two API instances running, creates at least 200 checks, pays all of them, cancels about 20%, **kills one API instance mid-run and restarts it**, waits for everything to settle, and then checks the POS:
   - every paid check that was not cancelled has exactly one open POS order,
   - every cancelled check has no open POS order,
   - no POS order exists that does not belong to one of our checks.

   We will run this ourselves, more than once.
4. **Normal tests** for the important logic. At least part of them must run against a real Postgres, not mocks.
5. `README.md`: how to run everything from a fresh clone in under 5 commands.
6. **Your Claude Code setup** and `AI_NOTES.md` (section 5).

Keep your full commit history. Do not squash it. We read it.

5. Claude Code setup (20 marks)

Set up the repo the way you would for a real production project where AI writes most of the code. We will clone it and use your setup ourselves. Include:

1. `CLAUDE.md`: what the project is, the commands, and the hard rules for *this* codebase. Short and specific beats long and generic. We mark down a pasted list of general best practices.
2. `.claude/settings.json` with:
   - **permissions**: what the agent may and may not do without asking,
   - **at least one hook that blocks something** (for example a `PreToolUse` hook that stops a dangerous action),
   - **at least one hook that checks the agent's work** (for example running type-check, lint or tests after edits or before the agent finishes).
3. **A git pre-commit hook** (Husky, Lefthook or similar) that runs the same checks your CI would run.
4. **Optional, counts as a bonus:** a custom subagent, skill or slash command you actually used (for example a reviewer that checks a diff against your rules), or a GitHub Actions workflow.
5. `AI_NOTES.md`, at most 1 page: three moments where Claude was wrong, or where you overruled it, and how you noticed. Include one hook or rule you added *because of* something that went wrong while you worked. Honest beats impressive.

In the README, say in one line what each hook does and why you added it. Every hook must work on a fresh clone.

6. How we mark it (100 marks)

Area	What we look at	Marks
Design and decision	The options you compared, how sound your reasoning is, the trade-offs you name, the risks you admit. `DECISIONS.md` carries most of this.	30
Correct under failure	The five guarantees hold. Your chaos test passes when we run it, repeatedly. We also try a few failure cases of our own.	25
Claude Code setup	`CLAUDE.md`, permissions, hooks, pre-commit, `AI_NOTES.md`. Does the setup actually stop bad code, or is it decoration?	20
Code quality and tests	Readable NestJS structure, strict TypeScript, a clean schema and migrations, tests that would fail if the code were broken.	15
Delivery	Runs from a fresh clone, a clear README, a readable commit history.	10

A smaller system that is correct and well argued scores higher than a bigger one that is not. You do not need a UI, authentication, real payments or a production deploy.

7. After you submit

We will book a 30–40 minute call. You will walk us through your design, defend it against the alternatives, and change one small thing live, with Claude, using your own setup. Be ready to explain every line you submit, including what the AI wrote.

Questions about the task are welcome. Ask early: unclear requirements are part of real work, and a good question counts in your favour. If you make an assumption instead, write it in `DECISIONS.md`.